

Hydroponik – Webanwendung zur pH-Überwachung

Stand: 24.09.2026

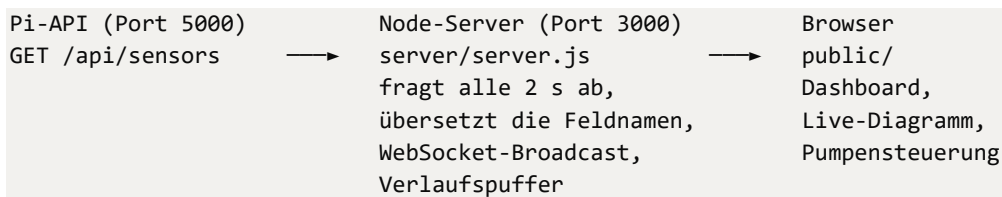
Dokumentation der Weboberfläche, der Direktverbindung zum Raspberry Pi über Ethernet und der Auswertung der Messreihe.

Gemessen wird der pH-Wert einer Hydroponik-Anlage, korrigiert wird mit einer Peristaltikpumpe für pH-Minus. Messung und Regelung laufen auf dem Raspberry Pi; diese Webanwendung zeigt die Werte **live im Browser** an und erlaubt manuelle Pumpentests. Sie ist bewusst **nicht** Teil des Regelkreises – fällt sie aus, misst und regelt der Pi unverändert weiter.

Inhalt

1. Aufbau der Webanwendung
2. Webanwendung starten
3. Direktverbindung zum Pi über Ethernet
4. Befehle auf dem Raspberry Pi
5. Messreihe auswerten – Kurven plotten
6. Fehlersuche

1 Aufbau der Webanwendung



iot/ – das Programm auf dem Pi

Datei	Aufgabe
hydroponik.py	Messprogramm mit pH-Regelung und Web-API (Port 5000). Läuft seit dem 17.09.2026 auf dem Pi; überarbeitete Fassung des Vorprojekt-Codes. Meldet pH-Wert, Sensorspannung in Volt, Nährlösungstemperatur, Betriebsart, Dosierzähler und Sperrzeit
kalibrieren.py	Zweipunktkalibrierung (pH 4,01 / 7,00) mit Verifikation bei pH 10,01. Schreibt <code>iot/kalibrierung.json</code> , die <code>hydroponik.py</code> beim Start einliest. Ohne diese Datei startet das Messprogramm nicht.
hydroponik.service	systemd-Dienst für den automatischen Start, bewusst mit <code>--nur-messen</code>
simulator.js	Ersatz für den Pi im Demobetrieb
archiv/	Frühere Programmstände, u. a. <code>ganzerCode.py</code> , mit dem alle Messreihen bis zum 11.08.2026 aufgenommen wurden (siehe <code>iot/archiv/README.md</code>)

Gemessen werden nur noch die Größen, die für die Regelung gebraucht werden: der pH-Wert über die Einstabmesskette und die Nährlösungstemperatur für die Kompensation. Lufttemperatur, Luftfeuchte und der Kontaktsensor aus dem Vorprojekt sind entfallen – sie erfüllten keine Anforderung, gingen in keine Entscheidung ein, und der DHT11 lieferte in rund drei Vierteln der Zyklen ohnehin keine Daten.

Auf dem Pi entstehen zwei Protokolle neben dem Skript: `hydroponik_log.csv` (jede Einzelmessung, alle 10 s) und `dosierungen_log.csv` (Zeitpunkt, Dauer und Menge jeder Dosierung). Die Pumpe ist im Sensor-Log praktisch nie als aktiv zu sehen, weil der Zustand während eines Laufs nicht aktualisiert wird; Dosierungen sind über das Dosierprotokoll belegt.

`server/server.js`

Der Node-Server hat vier Aufgaben:

Aufgabe	Beschreibung
Abfragen	Holt alle 2 s GET <code>/api/sensors</code> von der Pi-API (<code>PI_API_BASE_URL</code>)
Übersetzen	Bildet die Feldnamen des Pi (<code>wasser_temp</code> , <code>ph_spannung</code> , ...) auf das Datenmodell der Oberfläche ab; ungültige Messungen bleiben null und werden als -- angezeigt – es wird bewusst kein alter Wert weitergeführt
Verteilen	Schickt jeden neuen Messwert per WebSocket an alle verbundenen Browser
Puffern	Hält die Messwerte der letzten rund 24 Stunden (<code>HISTORY_LIMIT</code> , Standard 43200 Einträge bei 2-s-Takt) für CSV-Export und Diagramm; das Diagramm zeigt nach einem Seiten-Reload die letzten 120 Punkte

Schnittstellen des Node-Servers

Methode	Pfad	Zweck
GET	<code>/api/sensors</code>	letzter bekannter Zustand als JSON
GET	<code>/api/history</code>	pH-Verlauf für das Diagramm (<code>?limit=...</code> , Standard 120)
GET	<code>/api/export.csv</code>	gesamten Verlaufspuffer als CSV herunterladen
POST	<code>/api/sensors</code>	Zustand von außen setzen (Push statt Poll)
POST	<code>/api/pump/dose</code>	{ "seconds": 2 } → zeitbegrenzter Pumpenlauf am Pi
POST	<code>/api/pump/stop</code>	Not-Aus der Pumpe

Ein dauerhaftes Einschalten der Pumpe gibt es absichtlich nicht: der Pi kennt nur zeitbegrenzte Läufe, damit die Pumpe bei einem Verbindungsabbruch stoppt. POST `/api/pump/dose` antwortet erst nach Ablauf der Dosierzeit.

Umgebungsvariablen

Variable	Standard	Bedeutung
PORT	3000	Port der Weboberfläche

PI_API_BASE_URL	–	Basis-URL der Pi-API; leer = Demobetrieb
PI_SENSOR_POLL_INTERVAL_MS	2000	Abfrageintervall zum Pi
PUMP_MAX_SECONDS	10	Obergrenze je Pumpenlauf (der Pi begrenzt zusätzlich)
PH_TARGET / PH_TOLERANCE	5.8 / 0.2	Sollwert und Toleranz, falls der Pi sie nicht mitliefert; daraus folgt der Zielbereich 5,80 bis 6,00
HISTORY_LIMIT	43200	Größe des Verlaufspuffers (≈ 24 h bei 2-s-Takt); Diagramm zeigt die letzten 120

public/ – das Dashboard

Element	Funktion
Aktueller pH-Wert	Großanzeige mit Sensorspannung und Bewertung (im/über/unter Zielbereich)
Zielbereich	5,80 bis 6,00, also vom Sollwert bis zur Eingriffsschwelle. Bewusst unsymmetrisch: oberhalb von 6,00 dosiert die Regelung, unterhalb von 5,80 kann sie nichts ausrichten, da nur pH-Minus zur Verfügung steht
pH-Verlauf	Live-Liniendiagramm mit Zielband, Tooltip, Tastaturbedienung (Pfeiltasten) und Tabellenansicht; „CSV exportieren“ lädt die Messwerte herunter, „Diagramm speichern“ das Bild als PNG
Peristaltikpumpe	Betriebsart, Dosierzähler (gesamt und 24 h), Sperrzeit, Testlauf mit Sekundenangabe, Not-Aus; rote Warnung, wenn die Mengenbegrenzung ausgelöst hat
Nährlösungstemperatur	Wassertemperatur des Reservoirs, die über die Temperaturkompensation in den angezeigten pH-Wert eingeht

Die Verbindung läuft über WebSocket und baut sich nach einem Abbruch selbstständig neu auf. Zeigt die Kopfzeile „Getrennt“, ist der Node-Server aus – nicht der Pi.

2 Webanwendung starten

Einmalig, im Projektordner auf dem Windows-Rechner:

```
cd C:\Users\prati\hydroponic-system
npm install
```

Mit Raspberry Pi

```
.\start-pi.ps1
```

Das Skript prüft zuerst die eigene Ethernet-Adresse und die Erreichbarkeit des Pi, nennt bei einem Fehler die Ursache und startet dann den Server. Von Hand geht es auch:

```
$env:PI_API_BASE_URL = "http://192.168.10.1:5000"
npm start
```

Erfolgskontrolle im Terminal:

```
Pi-API: http://192.168.10.1:5000 (alle 2000 ms)
```

Dashboard: **http://localhost:3000** – die Betriebsart in der Pumpen-Karte muss messbetrieb oder regelung zeigen, nicht regelung (simuliert).

Ohne Pi (Demobetrieb)

```
npm start          # Terminal 1
npm run simulate    # Terminal 2
```

Der Simulator sendet dasselbe Datenformat wie der Pi, inklusive pH-Drift, Dosierung und Sperrzeit. Zum Vorführen der Mengenbegrenzung, ohne 20 Dosierungen abzuwarten:

```
$env:SIM_DOSE_LIMIT = "2"
npm run simulate
```

3 Direktverbindung zum Pi über Ethernet

Ein normales Patchkabel direkt zwischen Rechner und Pi – kein Router, kein Hochschulnetz dazwischen. Damit ist die Pumpen-Schnittstelle von außen nicht erreichbar. Beide Geräte bekommen feste Adressen im selben Subnetz:

Gerät	Adresse
Raspberry Pi (eth0)	192.168.10.1/24
Rechner (USB-Ethernet-Adapter „Ethernet 2“)	192.168.10.2/24

Wichtigste Fehlerquelle: das /24 am Ende. Ohne diese Angabe vergibt Linux /32 – eine Adresse ohne Netz darumherum. Der Pi hat dann keine Route zur Gegenseite, und der Ping meldet „Das Netzwerk ist nicht erreichbar“.

Windows-Seite (PowerShell **als Administrator**, einmalig)

```
Get-NetAdapter          # Adapternamen ablesen, hier: "Ethernet 2"
Set-NetIPInterface -InterfaceAlias "Ethernet 2" -Dhcp Disabled
New-NetIPAddress -InterfaceAlias "Ethernet 2" -IPAddress 192.168.10.2 -PrefixLength 24
```

Der Name in -InterfaceAlias muss exakt der Spalte Name aus Get-NetAdapter entsprechen. Dhcp Disabled verhindert, dass Windows auf dem Kabel vergeblich nach einem DHCP-Server sucht und sich selbst eine unbrauchbare 169.254-Adresse gibt.

Pi-Seite (einmalig)

Auf einem deutschsprachigen Raspberry Pi OS heißt das Ethernet-Profil „**Kabelgebundene Verbindung 1**“ (englisch: „Wired connection 1“) – den tatsächlichen Namen zeigt nmcli con show:

```
sudo nmcli con mod "Kabelgebundene Verbindung 1" ipv4.method manual ipv4.addresses
192.168.10.1/24
sudo nmcli con up "Kabelgebundene Verbindung 1"
ip -4 addr show eth0          # muss inet 192.168.10.1/24 zeigen
```

Diese Konfiguration überlebt Neustarts. **Nicht** dauerhaft auf sudo ip addr add ... setzen – das gilt nur bis zum nächsten Neustart und war in diesem Projekt die häufigste Ursache dafür, dass die Verbindung „plötzlich wieder weg“ war. Zum kurzen Ausprobieren ist es in Ordnung:

```
sudo ip addr add 192.168.10.1/24 dev eth0          # wirkt sofort, hält bis zum Neustart
```

Bewusst **ohne Gateway**: das Kabel dient nur der Messdatenübertragung. Das WLAN des Pi bleibt eingeschaltet – darüber kommen Updates und vor allem die Uhrzeit; ohne Zeitserver stimmen die Zeitstempel im CSV-Log nach jedem Neustart nicht, denn der Pi hat keine gepufferte Echtzeituhr.

Verbindung prüfen (Windows)

```
Test-NetConnection 192.168.10.1 -Port 5000
```

Feld	Sollwert	Wenn nicht ...
InterfaceAlias	Ethernet 2	steht dort WiFi, fehlt die Windows-Adresse (siehe oben)
PingSucceeded	True	Pi-Adresse prüfen: <code>ip -4 addr show eth0</code> → steht dort /24?
TcpTestSucceeded	True	läuft das Messprogramm auf dem Pi?

Täglicher Ablauf nach der Einrichtung

Kabel steckt, Pi einschalten, auf dem Rechner:

```
.\start-pi.ps1
```

Sonst nichts – vorausgesetzt, die feste IP ist per `nmcli` gesetzt und das Messprogramm startet automatisch (Abschnitt 4).

4 Befehle auf dem Raspberry Pi

Vor jeder Messreihe: Uhrzeit prüfen

```
date
```

Der Pi hat keine gepufferte Echtzeituhr. Ohne erreichbaren Zeitserver setzt er nach einem Neustart die zuletzt gespeicherte Uhrzeit fort – die Zeitstempel im Log tragen dann ein falsches Datum, nur die Zeitdifferenzen innerhalb einer Reihe stimmen. Das ist bei den Messreihen vom 07.08. und 11.08.2026 passiert (beide tragen Zeitstempel vom 06.08.). Stimmt die Zeit nicht, WLAN prüfen oder notfalls von Hand setzen:

```
sudo date -s "2026-09-18 14:30:00"
```

Kalibrieren (einmalig, und nach jedem Elektrodenwechsel oder Wässern)

```
cd ~/hydroponic-project
python3 iot/kalibrieren.py
```

Das Skript führt durch die Puffer pH 4,01, 7,00 und 10,01. Elektrode **und** Temperatursensor in den jeweiligen Becher halten, warten, bis der angezeigte Wert stabil ist, dann mit Enter bestätigen. Ergebnis: `iot/kalibrierung.json` mit Steilheit, Nullpunkt und Kalibriertemperatur. Liegt die Steilheit unter 85 % des theoretischen Werts, warnt das Skript (Anforderung M-06).

Messprogramm starten

```
cd ~/hydroponic-project
python3 iot/hydroponik.py --nur-messen      # messen und loggen, keine automatische
Dosierung
python3 iot/hydroponik.py                  # mit aktiver pH-Regelung
```

Im Messbetrieb bleibt der manuelle Testlauf über das Dashboard möglich. Die API bindet nur auf 192.168.10.1; zum Testen ohne Kabel HYDRO_API_HOST=0.0.0.0 voranstellen.

Läuft das Programm über SSH, beendet das Schließen des Fensters es mit. Für längere Messreihen deshalb tmux:

```
tmux new -s hydro
python3 iot/hydroponik.py --nur-messen
# Ablösen: Strg+B, dann D          Zurückholen: tmux attach -t hydro
```

Automatischer Start beim Hochfahren (empfohlen)

```
sudo cp iot/hydroponik.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now hydroponik

systemctl status hydroponik      # Zustand ansehen
journalctl -u hydroponik -f      # Ausgaben live mitlesen
sudo systemctl restart hydroponik # neu starten
```

Vorher in der Service-Datei User, WorkingDirectory und den Python-Pfad an die eigene Installation anpassen.

Dateien zwischen Rechner und Pi kopieren

```
# Windows → Pi (Skript aktualisieren)
scp C:\Users\prati\hydroponic-system\iot\hydroponik.py pi@192.168.10.1:~/hydroponic-project/iot/

# Pi → Windows (Messdaten holen; die Logs liegen neben dem Skript in iot/)
scp pi@192.168.10.1:~/hydroponic-project/iot/hydroponik_log.csv C:\Users\prati\hydroponic-system\
scp pi@192.168.10.1:~/hydroponic-project/iot/dosierungen_log.csv
C:\Users\prati\hydroponic-system\
```

Die CSV-Dateien **nicht mit Excel öffnen und wieder speichern**: Excel entfernt die Sekunden aus den Zeitstempeln und zerstört Dezimalzahlen (aus 5,45 wird „5.450.000.000.000.000“). Für die Auswertung immer die unveränderte Originaldatei verwenden.

Python-Abhängigkeiten installieren (einmalig)

```
python3 -m venv --system-site-packages ~/hydro-venv
source ~/hydro-venv/bin/activate
pip install -r iot/requirements.txt
```

5 Messreihe auswerten – Kurven plotten

Das Skript tools/plot_messreihe.py erzeugt aus der CSV-Logdatei druckfertige Diagramme (PNG, 300 dpi). Es liest sowohl das vollständige Log vom Pi (hydroponik_log.csv, per scp holen) als auch die über „CSV exportieren“ aus der Weboberfläche heruntergeladene Datei. Das Skript wertet timestamp, ph, ph_spannung und wasser_temp aus; weitere Spalten stören nicht, sodass auch ältere Messreihen lesbar bleiben.

Einrichtung (einmalig, auf dem Windows-Rechner)

```
pip install -r tools\requirements.txt
```

Aufrufe

```
# Standardauswertung: Zeitverlauf + Kalibrier-Kennlinie
python tools\plot_messreihe.py hydroponik_log.csv

# nur einen Zeitraum auswerten (z. B. die Sprungantwort nach einer Dosierung)
python tools\plot_messreihe.py hydroponik_log.csv --von "2026-08-05 14:00:00" --bis "2026-08-05 15:00:00"

# zusätzlich die Titrationskurve aus einer eigenen Messdatei
python tools\plot_messreihe.py hydroponik_log.csv --titration titration.csv

# anderer Zielbereich oder Dateiname
python tools\plot_messreihe.py hydroponik_log.csv --sollwert 6.0 --toleranz 0.3 --praefix versuch1
```

Die erzeugten Diagramme

Datei	Inhalt	Achsen
..._verlauf.png	pH-Wert und Wassertemperatur über der Zeit, untereinander mit gemeinsamer Zeitachse	x: Uhrzeit · y: pH-Wert bzw. Wassertemperatur in °C
..._kennlinie.png	Sensorspannung über pH mit Ausgleichsgerade; gibt Steilheit in mV/pH und R^2 aus	x: pH-Wert · y: Sensorspannung in V
..._titration.png	Dosis-Wirkungs-Kurve der pH-Minus-Lösung und deren Empfindlichkeit	x: Menge in ml/l bzw. pH-Wert · y: pH-Wert bzw. ΔpH je ml/l

Darstellungsregeln, die das Skript bereits umsetzt:

- **Keine zweite y-Achse.** pH und Temperatur stehen untereinander, weil die Zuordnung zweier Skalen in einem Bild willkürlich ist und Zusammenhänge vortäuscht.
- **Messlücken bleiben sichtbar.** Fällt der Sensor aus, reißt die Linie ab, statt die Lücke zu überbrücken.
- Der **pH-Wert trägt keine Einheit** (er ist als Logarithmus definiert); alle anderen Achsen folgen dem Muster „Größe in Einheit“ (DIN 461).
- Die Kennlinie muss eine Gerade mit **50–59 mV/pH** ergeben (Nernst-Bereich); das Skript warnt, wenn die Steilheit außerhalb liegt.

Format der Titrationsdatei

Von Hand angelegte CSV mit zwei Spalten – die zugegebene Menge auf das Lösungsvolumen normiert, damit das Ergebnis übertragbar ist:

```
volumen_ml_pro_l,ph
0.0,6.85
0.5,6.51
1.0,6.22
```

Für die Sprungantwort vorher im Pi-Skript `LOG_INTERVALL_S = 2` setzen – mit dem Standardwert (ein Logeintrag alle 30 s) ist die Auflösung für das Einschwingverhalten zu grob.

6 Fehlersuche

Symptom	Ursache	Lösung
Dashboard zeigt regelung (simuliert)	Simulator läuft statt Pi	Simulator beenden, <code>.\start-pi.ps1</code>
Pi-API nicht erreichbar: fetch failed	Pi aus, Adresse weg oder Messprogramm gestoppt	<code>Test-NetConnection 192.168.10.1 -Port 5000</code> , dann Tabelle in Abschnitt 3
Test-NetConnection zeigt InterfaceAlias: WiFi	Windows-Adresse fehlt → Pakete laufen ins WLAN	<code>New-NetIPAddress</code> ... als Administrator (Abschnitt 3)
Ping scheitert, obwohl beide Seiten konfiguriert sind	<code>/32</code> statt <code>/24</code> auf dem Pi	<code>ip -4 addr show eth0</code> prüfen, per <code>nmcli</code> mit <code>/24</code> neu setzen
„Access is denied“ bei <code>New-NetIPAddress</code>	PowerShell ohne Administratorrechte	Win+X → Terminal (Administrator)
„Unbekannte Verbindung“ bei <code>nmcli</code>	Profilname ist deutsch	<code>nmcli con show</code> , Namen in Anführungszeichen verwenden
Verbindung nach Pi-Neustart weg	Adresse war nur mit <code>ip addr add</code> gesetzt	dauerhaft per <code>nmcli</code> konfigurieren (Abschnitt 3)
<code>npm start</code> meldet „Kein <code>PI_API_BASE_URL</code> gesetzt“	Variable gilt nur im jeweiligen Fenster	<code>.\start-pi.ps1</code> verwenden
Port 3000 belegt	alter Node-Prozess läuft noch	<code>taskkill /F /IM node.exe</code> , dann neu starten
Pi: „Keine Kalibrierung gefunden“	<code>iot/kalibrierung.json</code> fehlt	<code>python3 iot/kalibrieren.py</code> ausführen (Abschnitt 4)
Pi: „API konnte nicht an 192.168.10.1:5000 binden“	Adresse beim Start noch nicht auf <code>eth0</code>	Programm versucht es alle 5 s erneut; feste Adresse per <code>nmcli</code> prüfen (Abschnitt 3)
Dashboard zeigt Sensorspannung -, pH aber normal	Pi liefert die Spannung nicht oder in mV	Programmstand auf dem Pi prüfen: <code>iot/hydroponik.py</code> meldet Volt
Zeitstempel im Log mit altem Datum	Pi ohne Zeitserver gestartet	<code>date</code> prüfen, Zeit setzen (Abschnitt 4); Zeitdifferenzen bleiben gültig
Testlauf/Not-Aus im Dashboard ohne Wirkung	auf dem Pi läuft <code>ganzerCode.py</code> (alte Endpunkte)	aktuelles <code>iot/hydroponik.py</code> auf den Pi kopieren
Skriptausführung von <code>start-pi.ps1</code> blockiert	PowerShell-Richtlinie	<code>Set-ExecutionPolicy -Scope Process -Bypass</code>